

AlbideyNet — Explication du code

Frontend Angular et backend Express/Prisma, expliqués fichier par fichier

Mariam Tchou Mity — INSTA Abéché

Septembre 2026

Introduction

Vue d'ensemble du projet

Frontend (Angular) — `src/app/`

Backend (Express + Prisma)

Frontend — `app.routes.ts`

Les imports

La liste des routes

Frontend — `services/auth.ts`

Les imports

Les formes des données

Le début du service

La connexion

Inscription et mot de passe oublié

Déconnexion et état de connexion

Profil

Frontend — `pages/home/home.ts`

Les variables de la page

La date du jour

Le chargement des chiffres au démarrage

La recherche

Frontend — `pages/home/home.css` (les pastilles du tableau de bord)

Frontend — `services/product.ts`

La forme d'un produit

Récupérer les produits

Créer, modifier, supprimer

Frontend — `pages/products/products.ts`

Les variables de la page

Les droits d'accès

Le filtre de recherche

Ouvrir le formulaire

L'image du produit

Enregistrer

La sélection multiple

L'affichage du prix

Backend — `index.js`

Mise en place

Réinitialisation de mot de passe et emails

Vérifier qu'on est bien connecté

Qui a le droit de faire quoi

Inscription

Connexion

Mot de passe oublié

Remplacer le mot de passe

Catégories (le modèle répété pour la plupart des données)

Produits (les particularités)

Mouvements de stock (la partie la plus complexe)

La Corbeille

Démarrage du serveur

Résumé

Introduction

Ce document reprend, fichier par fichier, l'explication du code de l'application **AlbideyNet** (gestion de stock), pour préparer la soutenance du projet de fin d'étude. Il couvre la structure générale du projet, puis le détail commenté des fichiers les plus représentatifs du frontend (Angular) et du backend (Express + Prisma).

Vue d'ensemble du projet

Frontend (Angular) — `src/app/`

Racine de l'application

- `app.ts`, `app.html`, `app.css` : le squelette général de l'application (barre de navigation, mise en page commune à toutes les pages).
- `app.routes.ts` : la liste des routes (quelle page s'affiche pour quelle adresse — `/login`, `/home`, `/products`, etc.).

`services/` — la logique partagée, indépendante de l'affichage

- `auth.ts` : connexion, inscription, mot de passe oublié.
- `product.ts`, `category.ts`, `supplier.ts`, `stock-movement.ts` : appels au serveur pour chaque type de donnée.
- `lang.ts` + `translations.ts` : gestion des langues (français / anglais / arabe).
- `theme.ts` : mode sombre / clair.
- `trash.ts`, `confirm-dialog.ts` : corbeille et boîtes de confirmation.

`pipes/`

- `translate.ts` : utilisé dans les pages pour afficher le bon texte selon la langue choisie.

`interceptors/`

- `auth-interceptor.ts` : ajoute automatiquement le jeton de connexion à chaque requête envoyée au serveur.

`pages/` — une page = un dossier avec 3 fichiers (`.ts` la logique, `.html` l'affichage, `.css` le style)

`login`, `register`, `forgot-password`, `reset-password`, `home` (le tableau de bord), `products`, `categories`, `suppliers`, `stock-movements`, `alerts`, `trash`, `profile`.

Backend (Express + Prisma)

Le serveur est beaucoup plus simple côté organisation : un seul fichier `index.js` (dans le dossier du projet backend) qui contient toutes les routes de l'API, avec **Prisma** comme intermédiaire pour parler à la base de données **PostgreSQL**.

C'est le fichier le plus simple : un aiguillage entre les adresses (URL) et les pages.

Les imports

```
import { Routes } from '@angular/router';
import { Login } from '../pages/login/login';
import { Register } from '../pages/register/register';
import { ForgotPassword } from '../pages/forgot-password/forgot-password';
import { ResetPassword } from '../pages/reset-password/reset-password';
import { Home } from '../pages/home/home';
import { Products } from '../pages/products/products';
import { Categories } from '../pages/categories/categories';
import { Suppliers } from '../pages/suppliers/suppliers';
import { StockMovements } from '../pages/stock-movements/stock-movements';
import { Alerts } from '../pages/alerts/alerts';
import { Profile } from '../pages/profile/profile';
import { Trash } from '../pages/trash/trash';
```

`import { Routes } from '@angular/router'` amène l'outil d'Angular qui gère les routes. Les autres lignes amènent chaque composant de page pour pouvoir l'utiliser plus bas.

La liste des routes

```
export const routes: Routes = [
  { path: 'login', component: Login },
  { path: 'register', component: Register },
  { path: 'forgot-password', component: ForgotPassword },
  { path: 'reset-password', component: ResetPassword },
  { path: 'home', component: Home },
  { path: 'products', component: Products },
  { path: 'categories', component: Categories },
  { path: 'suppliers', component: Suppliers },
  { path: 'stock-movements', component: StockMovements },
  { path: 'alerts', component: Alerts },
  { path: 'profile', component: Profile },
  { path: 'trash', component: Trash },
  { path: '', redirectTo: '/login', pathMatch: 'full' }
];
```

`export const routes` crée une liste que le reste de l'application utilise. Chaque ligne, comme `{ path: 'login', component: Login }`, dit : “quand l'adresse est `/login`, affiche le composant `Login`”. La dernière ligne redirige automatiquement vers `/login` quand l'adresse est vide — c'est pour ça que l'application ouvre toujours sur la page de connexion.

Le service qui gère tout ce qui touche au compte utilisateur : connexion, inscription, mot de passe oublié, profil.

Les imports

```
import { Injectable, signal } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, tap } from 'rxjs';
```

`HttpClient` sert à envoyer des requêtes au serveur. `Injectable` permet à cette classe d'être "un service" partagé par toute l'application. `signal` est la façon moderne dont Angular garde une valeur qui peut changer (ici : l'utilisateur connecté) et prévient automatiquement l'affichage quand elle change. `Observable` et `tap` viennent de RxJS, une bibliothèque qui gère les résultats qui arrivent avec un délai (la réponse du serveur).

Les formes des données

```
export interface AppUser {
  id: number;
  name: string;
  email: string;
  role: string;
  phone?: string | null;
  address?: string | null;
  avatarUrl?: string | null;
}

export interface LoginResponse {
  token: string;
  user: AppUser;
}
```

Une `interface` en TypeScript décrit "à quoi doit ressembler" une donnée, sans exécuter de code. `AppUser` décrit un utilisateur ; `LoginResponse` décrit ce que le serveur renvoie après une connexion réussie.

Le début du service

```
@Injectable({ providedIn: 'root' })
export class Auth {
  private apiUrl = 'http://localhost:3000/api/auth';
  currentUser = signal<AppUser | null>(null);

  constructor(private http: HttpClient) {
    const saved = localStorage.getItem('user');
    if (saved) this.currentUser.set(JSON.parse(saved));
  }
}
```

`@Injectable({ providedIn: 'root' })` dit à Angular de créer une seule copie de ce service pour toute l'application. `currentUser` retient qui est connecté (`null` par défaut). Le `constructor` vérifie, au démarrage, si un utilisateur était déjà enregistré dans `localStorage` (la mémoire du navigateur qui survit à la fermeture de l'onglet) — c'est ce qui permet de rester connectée après avoir rechargé la page.

La connexion

```
login(email: string, password: string): Observable<LoginResponse> {
  return this.http.post<LoginResponse>(`${this.apiUrl}/login`, { email, password }).pipe(
    tap((res) => {
      localStorage.setItem('token', res.token);
      localStorage.setItem('user', JSON.stringify(res.user));
      this.currentUser.set(res.user);
    })
  );
}
```

`this.http.post(...)` envoie l'email et le mot de passe au serveur. `tap(...)` s'exécute quand la réponse arrive : on sauvegarde le `token` (la preuve de connexion) et les infos de l'utilisateur, et on met à jour `currentUser`.

Inscription et mot de passe oublié

```

register(data: { name: string; email: string; password: string }): Observable<AppUser> {
  return this.http.post<AppUser>(`${this.apiUrl}/register`, data);
}

forgotPassword(email: string): Observable<{ message: string }> {
  return this.http.post<{ message: string }>(`${this.apiUrl}/forgot-password`, { email });
}

resetPassword(token: string, newPassword: string): Observable<{ message: string }> {
  return this.http.post<{ message: string }>(`${this.apiUrl}/reset-password`, { token, newPassword });
}

```

Chacune envoie une information au serveur et attend une réponse : les infos d'inscription, l'email pour la réinitialisation, ou le nouveau mot de passe avec le jeton reçu par email.

Déconnexion et état de connexion

```

logout(): void {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
  this.currentUser.set(null);
}

getToken(): string | null {
  return localStorage.getItem('token');
}

isLoggedIn(): boolean {
  return !!this.getToken();
}

```

`logout()` efface le token et l'utilisateur mémorisés. `isLoggedIn()` répond vrai ou faux selon qu'un jeton existe (!! transforme n'importe quelle valeur en `true / false`) — c'est ce que l'application utilise pour savoir si elle doit laisser accéder aux pages protégées.

Profil

```

getMe(): Observable<AppUser> {
  return this.http.get<AppUser>(`${this.apiUrl}/me`).pipe(
    tap((user) => {
      localStorage.setItem('user', JSON.stringify(user));
      this.currentUser.set(user);
    })
  );
}

updateProfile(data: { name?: string; email?: string; phone?: string | null; avatarUrl?: string | null }):
Observable<AppUser> {
  return this.http.put<AppUser>(`${this.apiUrl}/profile`, data).pipe(
    tap((user) => {
      localStorage.setItem('user', JSON.stringify(user));
      this.currentUser.set(user);
    })
  );
}

```

`getMe()` redemande au serveur les infos à jour de l'utilisateur connecté. `updateProfile()` envoie les changements du profil et met à jour `currentUser`.

La logique derrière le tableau de bord : elle récupère les chiffres affichés dans les pastilles (produits, catégories, fournisseurs, mouvements, alertes).

Les variables de la page

```
export class Home implements OnInit {
  productCount = 0;
  categoryCount = 0;
  supplierCount = 0;
  movementCount = 0;
  lowStockCount = 0;
  loading = true;

  searchProducts = '';
  searchCategories = '';
  searchSuppliers = '';
  searchMovements = '';
  searchAlerts = '';
```

Ce sont exactement les chiffres affichés dans les pastilles rouge/marine. `loading = true` sert à afficher “Chargement...” tant que les vraies données n’ont pas encore été reçues.

La date du jour

```
get today(): string {
  const formatted = new Intl.DateTimeFormat(this.lang.locale, { weekday: 'long', day: 'numeric', month: 'long', year: 'numeric' }).format(new Date());
  return formatted.charAt(0).toUpperCase() + formatted.slice(1);
}
```

`Intl.DateTimeFormat` est un outil intégré au navigateur qui écrit une date dans le bon format selon la langue. La dernière ligne met la première lettre en majuscule.

Le chargement des chiffres au démarrage

```
ngOnInit(): void {
  let loaded = 0;
  const total = 5;
  const checkDone = () => {
    loaded++;
    if (loaded === total) {
      this.loading = false;
      this.cdr.detectChanges();
    }
  };

  this.productService.getAll().subscribe({
    next: (data) => { this.productCount = data.length; checkDone(); },
    error: () => checkDone()
  });

  // ... même principe pour categoryService, supplierService,
  // stockMovementService, et productService.getLowStock()
}
```

`ngOnInit` s’exécute automatiquement dès que la page s’ouvre. Comme il y a 5 informations à récupérer, `loaded` compte combien sont arrivées ; dès que les 5 sont là, on arrête d’afficher “Chargement...”. Pour chaque service, `.getAll().subscribe(...)` demande la liste complète au serveur, et `data.length` (le nombre d’éléments dans la liste) devient le chiffre affiché.

La recherche

```
goToSearch(path: string, term: string): void {
  const value = (term || '').trim();
  if (value) {
    this.router.navigate([path], { queryParams: { search: value } });
  } else {
    this.router.navigate([path]);
  }
}
```

C’est ce qui s’exécute quand on tape dans une barre de recherche d’une carte et qu’on appuie sur Entrée ou clique sur la loupe : ça envoie vers la page correspondante, avec la recherche déjà pré-remplie si quelque chose a été tapé.

```
.stat-info {
  display: inline-flex;
  flex-direction: column;
  align-items: center;
  gap: 2px;
  background: #1e3a5f;
  border-radius: 8px;
  padding: 5px 12px;
  margin-bottom: 6px;
  box-shadow: 0 3px 10px rgba(0, 0, 0, 0.3);
}

.stat-value {
  font-size: 20px;
  font-weight: 700;
  line-height: 1;
  color: #ef4444;
}

.stat-label {
  font-size: 10.5px;
  font-weight: 600;
  white-space: nowrap;
  color: #f1f5f9;
}
```

`.stat-info` est la pastille elle-même : `background: #1e3a5f` est sa couleur marine, `border-radius` arrondit les coins, `box-shadow` ajoute une ombre pour la détacher visuellement de l'image derrière. `.stat-value` est le chiffre, en rouge (`color: #ef4444`) et en gras. `.stat-label` est le texte en dessous, en blanc cassé (`#f1f5f9`).

Le service qui gère tout ce qui touche aux produits ; `category.ts`, `supplier.ts` et `stock-movement.ts` sont construits sur le même principe.

La forme d'un produit

```
export interface Product {
  id: number;
  name: string;
  sku: string;
  description: string | null;
  imageUrl?: string | null;
  price: number;
  quantity: number;
  minQuantity: number;
  categoryId: number | null;
  supplierId: number | null;
  category?: { id: number; name: string };
  supplier?: { id: number; name: string };
}
```

Le `?` après `imageUrl`, `category` et `supplier` veut dire que ces informations sont optionnelles — un produit peut exister sans image, par exemple.

Récupérer les produits

```
@Injectable({ providedIn: 'root' })
export class ProductService {
  private apiUrl = 'http://localhost:3000/api/products';

  constructor(private http: HttpClient) {}

  getAll(): Observable<Product[]> {
    return this.http.get<Product[]>(this.apiUrl);
  }

  getLowStock(): Observable<Product[]> {
    return this.http.get<Product[]>(`${this.apiUrl}/low-stock`);
  }
}
```

`getAll()` demande la liste complète des produits. `getLowStock()` demande seulement les produits en stock faible (le serveur fait le tri) — c'est elle qui alimente la carte "Alertes stock faible" et la page Alertes.

Créer, modifier, supprimer

```
create(product: Partial<Product>): Observable<Product> {
  return this.http.post<Product>(this.apiUrl, product);
}

update(id: number, product: Partial<Product>): Observable<Product> {
  return this.http.put<Product>(`${this.apiUrl}/${id}`, product);
}

delete(id: number): Observable<void> {
  return this.http.delete<void>(`${this.apiUrl}/${id}`);
}
}
```

`Partial<Product>` veut dire "un produit, mais pas obligé d'avoir tous les champs remplis" — utile pour un formulaire. Ces trois fonctions sont appelées par la page Produits quand on clique sur "Enregistrer", "Modifier" ou l'icône poubelle.

La page complète de gestion des produits : liste, recherche, formulaire, sélection multiple, suppression.

Les variables de la page

```
export class Products implements OnInit {
  products: Product[] = [];
  categories: Category[] = [];
  suppliers: Supplier[] = [];
  loading = true;
  error = '';
  saveError = '';
  searchTerm = '';
  showForm = false;
  editingProduct: Product | null = null;
  formData: Partial<Product> = this.emptyForm();
}
```

`showForm` dit si le formulaire d’ajout/modification est affiché. `editingProduct` retient quel produit est en train d’être modifié (`null` si on crée un nouveau produit).

Les droits d’accès

```
get canManage(): boolean {
  const role = this.auth.currentUser()?.role;
  return role === 'ADMIN' || role === 'DIRECTEUR_GENERAL' || role === 'RESPONSABLE_LOGISTIQUE';
}
```

Vérifie le rôle de la personne connectée et répond vrai seulement pour Admin, Directeur Général ou Responsable Logistique — cette valeur décide si les boutons “Nouveau produit” ou “Supprimer” sont visibles.

Le filtre de recherche

```
get filteredProducts(): Product[] {
  const term = this.searchTerm.trim().toLowerCase();
  if (!term) return this.products;
  return this.products.filter(p =>
    p.name?.toLowerCase().includes(term) ||
    p.sku?.toLowerCase().includes(term) ||
    p.description?.toLowerCase().includes(term) ||
    p.category?.name?.toLowerCase().includes(term)
  );
}
```

C’est la vraie liste affichée à l’écran. Si la recherche est vide, on montre tout ; sinon, on ne garde que les produits dont le nom, le SKU, la description ou la catégorie contient le texte tapé.

Ouvrir le formulaire

```
openCreateForm(): void {
  this.editingProduct = null;
  this.formData = this.emptyForm();
  this.showForm = true;
}

openEditForm(product: Product): void {
  this.editingProduct = product;
  this.formData = { name: product.name, sku: product.sku, /* ... */ };
  this.showForm = true;
}
```

`openCreateForm()` (bouton “Nouveau produit”) vide le formulaire. `openEditForm(product)` (bouton crayon) le remplit avec les infos du produit cliqué.

L’image du produit

```
onImageSelected(event: Event): void {
  const file = input.files && input.files[0];
  if (!file.type.startsWith('image/')) { this.saveError = /* ... */; return; }
  if (file.size > 3 * 1024 * 1024) { this.saveError = /* ... */; return; }
  const reader = new FileReader();
  reader.onload = () => { this.formData.imageUrl = reader.result as string; };
  reader.readAsDataURL(file);
}
```

On vérifie que le fichier est bien une image et qu'il ne dépasse pas 3 Mo. `FileReader` convertit ensuite l'image en texte encodé que le formulaire peut envoyer.

Enregistrer

```
saveProduct(): void {
  if (this.editingProduct) {
    this.productService.update(this.editingProduct.id, this.formData).subscribe({ /* ... */ });
  } else {
    this.productService.create(this.formData).subscribe({ /* ... */ });
  }
}
```

Une seule fonction pour les deux cas : si `editingProduct` existe, on modifie ; sinon on crée. Dans les deux cas, si ça réussit, on referme le formulaire et on recharge la liste.

La sélection multiple

```
selectedIds = new Set<number>();

toggleSelect(item: { id: number }): void {
  if (this.selectedIds.has(item.id)) {
    this.selectedIds.delete(item.id);
  } else {
    this.selectedIds.add(item.id);
  }
}

async deleteSelected(): Promise<void> {
  const confirmed = await this.confirmDialog.confirm({ /* ... */ });
  if (!confirmed) return;
  // supprime un par un tous les produits sélectionnés
}
```

`selectedIds` est un `Set` (collection sans doublons) qui retient les identifiants des produits cochés. `deleteSelected()` demande d'abord confirmation, puis supprime chaque produit sélectionné.

L'affichage du prix

```
formatPrice(price: number | undefined | null): string {
  if (price === undefined || price === null) return '0 FCFA';
  return Math.round(price).toLocaleString('fr-FR') + ' FCFA';
}
```

Transforme un nombre comme `15000` en `"15 000 FCFA"` — `toLocaleString('fr-FR')` ajoute les espaces entre les milliers, au format français.

Le serveur de l'application, écrit avec **Express** (pour recevoir les requêtes) et **Prisma** (pour parler à la base de données PostgreSQL). Il est organisé en blocs répétés pour chaque type de donnée.

Mise en place

```
const express = require("express");
const { PrismaClient } = require("@prisma/client");
const bcrypt = require("bcryptjs");
const jwt = require("jsonwebtoken");
const crypto = require("crypto");
const nodemailer = require("nodemailer");
require("dotenv").config();
const cors = require("cors");
const app = express();
app.use(cors());
const prisma = new PrismaClient();
const PORT = process.env.PORT || 3000;
const JWT_SECRET = process.env.JWT_SECRET || "changez-cette-cle-en-production";
```

`express` est le cœur du serveur. `PrismaClient` parle à la base de données. `bcryptjs` chiffre les mots de passe (jamais stockés en clair). `jsonwebtoken` crée les jetons de connexion. `nodemailer` envoie les emails. `cors` autorise l'application Angular (sur une autre adresse) à parler à ce serveur. `require("dotenv").config()` lit le fichier `.env` pour récupérer les informations secrètes sans les écrire dans le code.

Réinitialisation de mot de passe et emails

```
const resetTokens = new Map();

const mailTransporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST,
  port: Number(process.env.SMTP_PORT) || 587,
  secure: Number(process.env.SMTP_PORT) === 465,
  auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS },
});

app.use(express.json({ limit: "10mb" }));
```

`resetTokens` retient temporairement quel jeton correspond à quel email. `mailTransporter` est la boîte d'envoi configurée avec les identifiants Gmail. La dernière ligne accepte les requêtes au format JSON jusqu'à 10 Mo (à cause des images de produits envoyées en base64).

Vérifier qu'on est bien connecté

```
function authMiddleware(req, res, next) {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith("Bearer ")) {
    return res.status(401).json({ error: "Token manquant" });
  }
  const token = authHeader.split(" ")[1];
  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = decoded;
    next();
  } catch (err) {
    return res.status(401).json({ error: "Token invalide ou expiré" });
  }
}
```

C'est un "videur à l'entrée" : avant certaines routes, cette fonction vérifie que la requête contient un jeton valide. Si absent ou invalide, on refuse (401). Sinon, `jwt.verify` retrouve les infos de l'utilisateur mises dans le jeton à la connexion, et `next()` laisse passer la requête.

Qui a le droit de faire quoi

```

const ROLES_GESTION = ["ADMIN", "DIRECTEUR_GENERAL", "RESPONSABLE_LOGISTIQUE"];
const ROLES_LECTURE_SEULE = ["AGENT_LOGISTIQUE"];

function canManage(req, res, next) {
  if (!ROLES_GESTION.includes(req.user.role)) {
    return res.status(403).json({ error: ".." });
  }
  next();
}

function canWrite(req, res, next) {
  if (ROLES_LECTURE_SEULE.includes(req.user.role)) {
    return res.status(403).json({ error: ".." });
  }
  next();
}

```

`canManage` bloque tout le monde sauf Admin, Directeur Général et Responsable Logistique (pour supprimer ou gérer la Corbeille).
`canWrite` bloque uniquement les Agents Logistiques, qui n'ont le droit que de consulter.

Inscription

```

app.post("/api/auth/register", async (req, res) => {
  try {
    const { name, email, password, phone, address, avatarUrl } = req.body;
    if (!name || !email || !password) {
      return res.status(400).json({ error: "Nom, email et mot de passe requis" });
    }
    const existing = await prisma.user.findUnique({ where: { email } });
    if (existing) {
      return res.status(409).json({ error: "Cet email est déjà utilisé" });
    }
    const hashedPassword = await bcrypt.hash(password, 10);
    const user = await prisma.user.create({
      data: { name, email, password: hashedPassword, role: "EMPLOYE", phone, address, avatarUrl },
    });
    res.status(201).json({ id: user.id, name: user.name, email: user.email, role: user.role });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

On vérifie que les champs obligatoires sont remplis, puis que l'email n'est pas déjà pris. `bcrypt.hash(password, 10)` transforme le mot de passe en une version chiffrée illisible — c'est cette version qui est stockée en base, jamais le mot de passe original. `prisma.user.create(...)` enregistre le nouvel utilisateur avec le rôle "EMPLOYE" par défaut.

Connexion

```

app.post("/api/auth/login", async (req, res) => {
  const { email, password } = req.body;
  const user = await prisma.user.findUnique({ where: { email } });
  if (!user) return res.status(401).json({ error: "Identifiants invalides" });

  const valid = await bcrypt.compare(password, user.password);
  if (!valid) return res.status(401).json({ error: "Identifiants invalides" });

  const token = jwt.sign(
    { id: user.id, email: user.email, role: user.role },
    JWT_SECRET,
    { expiresIn: "8h" }
  );
  res.json({ token, user: { id: user.id, name: user.name, email: user.email, role: user.role } });
});

```

`bcrypt.compare(password, user.password)` compare le mot de passe tapé avec la version chiffrée stockée — c'est le seul moyen de vérifier un mot de passe chiffré. `jwt.sign(...)` fabrique le jeton : un texte signé numériquement contenant l'id, l'email et le rôle, valable 8 heures.

Mot de passe oublié

```
app.post("/api/auth/forgot-password", async (req, res) => {
  const { email } = req.body;
  const user = await prisma.user.findUnique({ where: { email } });
  if (user) {
    const token = crypto.randomBytes(32).toString("hex");
    resetTokens.set(token, { email: user.email, expiresAt: Date.now() + 60 * 60 * 1000 });
    const resetUrl = `${process.env.FRONTEND_URL}/reset-password?token=${token}`;
    await mailTransporter.sendMail({
      to: user.email,
      subject: "Réinitialisation de votre mot de passe AlbideyNet",
      html: `<p>Cliquez sur ce lien : <a href="${resetUrl}">${resetUrl}</a></p>`,
    });
  }
  res.json({ message: "Si un compte existe avec cet email, un lien de réinitialisation a été envoyé." });
});
```

`crypto.randomBytes(32).toString("hex")` fabrique un jeton aléatoire impossible à deviner, valable 1 heure. Détail important : **la réponse est toujours la même**, qu'un compte existe ou non, pour ne pas révéler quels emails sont inscrits.

Remplacer le mot de passe

```
app.post("/api/auth/reset-password", async (req, res) => {
  const { token, newPassword } = req.body;
  const entry = resetTokens.get(token);
  if (!entry || entry.expiresAt < Date.now()) {
    resetTokens.delete(token);
    return res.status(400).json({ error: "Lien invalide ou expiré." });
  }
  const hashedPassword = await bcrypt.hash(newPassword, 10);
  await prisma.user.update({ where: { email: entry.email }, data: { password: hashedPassword } });
  resetTokens.delete(token);
  res.json({ message: "Mot de passe réinitialisé avec succès" });
});
```

On vérifie que le jeton existe et n'a pas expiré, on chiffre le nouveau mot de passe, on met à jour la base, puis on supprime le jeton pour qu'il ne puisse pas être réutilisé.

Catégories (le modèle répété pour la plupart des données)

```
app.get("/api/categories", authMiddleware, async (req, res) => {
  const categories = await prisma.category.findMany({
    where: { deletedAt: null },
    include: { products: { where: { deletedAt: null } } },
  });
  res.json(categories);
});

app.post("/api/categories", authMiddleware, canWrite, async (req, res) => {
  const category = await prisma.category.create({ data: { name: req.body.name } });
  res.status(201).json(category);
});

app.put("/api/categories/:id", authMiddleware, canWrite, async (req, res) => {
  const category = await prisma.category.update({
    where: { id: Number(req.params.id) },
    data: { name: req.body.name },
  });
  res.json(category);
});

app.delete("/api/categories/:id", authMiddleware, canManage, async (req, res) => {
  await prisma.category.update({
    where: { id: Number(req.params.id) },
    data: { deletedAt: new Date() },
  });
  res.status(204).send();
});
```

`where: { deletedAt: null }` filtre les catégories non supprimées. `:id` dans l'adresse est un paramètre (`/api/categories/5` → `req.params.id` vaut `"5"`). La suppression n'est **pas** définitive : on remplit `deletedAt` avec la date actuelle — c'est une "suppression douce" (*soft delete*) : la catégorie reste en base mais n'apparaît plus dans les listes normales, elle va dans la Corbeille.

```

app.post("/api/categories/:id/restore", authMiddleware, canManage, async (req, res) => {
  const category = await prisma.category.update({
    where: { id: Number(req.params.id) },
    data: { deletedAt: null },
  });
  res.json(category);
});

app.delete("/api/categories/:id/permanent", authMiddleware, canManage, async (req, res) => {
  await prisma.category.delete({ where: { id: Number(req.params.id) } });
  res.status(204).send();
});

```

restore remet `deletedAt` à `null`. La suppression définitive utilise `prisma.category.delete(...)` (et non `.update`), qui efface vraiment la ligne — utilisée par le bouton “Supprimer définitivement” de la Corbeille. Prisma refuse si des produits sont encore liés à cette catégorie (code d’erreur “P2003”), pour ne pas casser les données.

Les **fournisseurs** suivent exactement la même structure.

Produits (les particularités)

```

app.get("/api/products/low-stock", authMiddleware, async (req, res) => {
  const products = await prisma.product.findMany({
    where: { deletedAt: null },
    include: { category: true, supplier: true },
  });
  const lowStock = products.filter((p) => p.quantity <= p.minQuantity);
  res.json(lowStock);
});

```

Prisma ne peut pas comparer directement deux colonnes (`quantity` et `minQuantity`) dans sa requête ; on récupère donc tous les produits puis on filtre nous-mêmes en JavaScript. C’est cette liste qui alimente la pastille “Alertes stock faible” et la page Alertes.

```

app.post("/api/products", authMiddleware, canWrite, async (req, res) => {
  const { name, sku, description, price, quantity, minQuantity, categoryId, supplierId, imageUrl } = req.body;
  const product = await prisma.product.create({
    data: {
      name, sku, description, imageUrl,
      price: Number(price),
      quantity: Number(quantity) || 0,
      minQuantity: Number(minQuantity) || 5,
      categoryId: categoryId ? Number(categoryId) : null,
      supplierId: supplierId ? Number(supplierId) : null,
    },
  });
  res.status(201).json(product);
});

```

Chaque champ numérique est explicitement converti avec `Number(...)`, avec une valeur de secours (`|| 0`) si le champ arrive vide — une sécurité pour ne jamais planter.

Mouvements de stock (la partie la plus complexe)

```

app.post("/api/movements", authMiddleware, canWrite, async (req, res) => {
  const { productId, type, quantity, reason, date } = req.body;
  if (!["ENTREE", "SORTIE"].includes(type)) {
    return res.status(400).json({ error: "Type doit être ENTREE ou SORTIE" });
  }
  const result = await prisma.$transaction(async (tx) => {
    const product = await tx.product.findUnique({ where: { id: Number(productId) } });
    const newQuantity =
      type === "ENTREE" ? product.quantity + Number(quantity) : product.quantity - Number(quantity);
    if (newQuantity < 0) throw new Error("Stock insuffisant pour cette sortie");
    await tx.product.update({ where: { id: product.id }, data: { quantity: newQuantity } });
    return tx.stockMovement.create({
      data: { productId: product.id, type, quantity: Number(quantity), reason, userId: req.user.id },
    });
  });
  res.status(201).json(result);
});

```

`prisma.$transaction(async (tx) => { ... })` regroupe deux opérations liées — modifier la quantité du produit ET créer l’enregistrement du mouvement — pour qu’elles réussissent **toutes les deux ensemble**, ou échouent toutes les deux. Sans cette précaution, une panne entre les deux opérations laisserait le stock incohérent avec l’historique. La vérification `if (newQuantity < 0)` empêche de faire sortir plus de stock qu’il n’en existe.

Modifier ou supprimer un mouvement suit la même logique mais doit d’abord **annuler l’effet de l’ancien mouvement** sur le stock avant d’appliquer le nouveau (ou de le retirer), toujours à l’intérieur d’une transaction.

La Corbeille

```
app.get("/api/trash", authMiddleware, canManage, async (req, res) => {
  const [products, categories, suppliers, movements] = await Promise.all([
    prisma.product.findMany({ where: { deletedAt: { not: null } } }),
    prisma.category.findMany({ where: { deletedAt: { not: null } } }),
    prisma.supplier.findMany({ where: { deletedAt: { not: null } } }),
    prisma.stockMovement.findMany({ where: { deletedAt: { not: null } } }),
  ]);
  res.json({ products, categories, suppliers, movements });
});
```

`deletedAt: { not: null }` est l'inverse du filtre habituel : on veut seulement les éléments qui ont une date de suppression.
`Promise.all([...])` lance les 4 requêtes en même temps plutôt que l'une après l'autre, pour aller plus vite.

Démarrage du serveur

```
app.listen(PORT, () => {
  console.log(`Serveur AlbideyNet démarré sur http://localhost:${PORT}`);
});
```

La dernière étape : le serveur commence à écouter les requêtes sur le port défini (3000 par défaut), et affiche un message de confirmation dans le terminal.

Résumé

L'application AlbideyNet suit une architecture classique en deux parties :

- Le **frontend** (Angular) est organisé par pages (une page = logique `.ts` + affichage `.html` + style `.css`), avec des **services** partagés pour parler au serveur et des **pipes/interceptors** transverses (traduction, jeton d'authentification).
- Le **backend** (Express + Prisma) expose une API avec des routes organisées par type de donnée, protégées par deux niveaux de droits (`canWrite` pour la lecture seule des Agents Logistiques, `canManage` pour la suppression réservée aux rôles de direction), et utilise la **suppression douce** (`deletedAt`) pour permettre la Corbeille et la restauration. Les **mouvements de stock** sont la partie la plus délicate, protégée par des **transactions** pour garder le stock et son historique toujours cohérents.